



stC: The Strongly-Typed C Programming Language

Germán Rivera

`jgrivera67@gmail.com`

March 16, 2026

Contents

1	Introduction	1
1.1	Language Overview	1
1.2	stcc	3
2	Lexical Elements	4
2.1	Tokens	4
2.2	Keywords	4
2.3	Identifiers	5
2.4	Literals	5
2.4.1	Integer Literals	5
2.4.2	Floating-Point Literals	5
2.4.3	Character Literals	6
2.4.4	String Literals	6
2.4.5	Boolean Literals	6
2.5	Operators	6
2.6	Comments	7
2.7	Built-in Constants	7
2.7.1	Platform Constants	7
2.7.2	Compiler Options	7
3	Examples	9
3.1	Simple Program	9
3.2	Scalar Types	9
3.2.1	Range Types	9
3.2.2	Modular Types	10
3.2.3	Floating Point Types	11
3.2.4	Fixed Point Types	12
3.2.5	Subtypes	13
3.2.6	Structure Types	14
3.2.7	Structure Types with Type Invariants	14
3.2.8	Union Types	15
3.2.9	Enumeration Types	15

3.2.10	Bit Fields and Memory-Mapped I/O Registers	15
3.3	Arrays	18
3.4	Functions	18
3.4.1	Attributes and Aspects	18
3.4.2	Simple Function	19
3.4.3	Function with Contracts	19
3.4.4	Function with Global-Effect Contracts	19
3.4.5	Function with Parameter Modes	20
3.5	Control Flow	20
3.5.1	If Statement	20
3.5.2	While Loop	20
3.5.3	While Loop with Loop Contracts	20
3.5.4	For Loop	21
3.5.5	Do-While Loop	21
3.5.6	Switch Statement	21
3.6	Identifier Attributes	22
3.6.1	Range Type Attributes	22
3.6.2	Array Attributes	22
3.6.3	Available Attributes	23
3.7	Sizeof Operator	23
3.8	Expressions	24
3.8.1	Ternary Conditional	24
3.8.2	Range Operator	24
3.9	Pointers	24
3.10	Function Signature Types	25
3.11	Module Example	25
3.12	Child Modules	26
3.12.1	Private Child Modules	26
3.12.2	Private Imports	27
3.13	Compile-Time Conditionals	28
3.13.1	Platform-Specific Types	28
3.13.2	Numeric Literal Separators	29
3.14	Depends Contracts	29
3.15	Abstract State and Refined State	30
3.16	Generic Modules	30
3.16.1	Type and Function Formals	30
3.16.2	Type and Const Formals	31

A *stC* Grammar

32

Chapter 1

Introduction

Although most embedded software developers recognize the importance of strong typing, as a tool to write more reliable software, many get discouraged of using Ada, the strongest-typed language there is, just because of its syntactic verbosity, or perhaps simply because Ada does not look more like C.

What if we could have a new programming language for embedded software that looks like C but has an Ada soul, that is, a language with all the Ada concepts but in a C skin, C with a strongly-typed armor, and where using pointers is more the exception than the norm.

This document presents *stC* (*Strongly-Typed C*), a programming language for safety-critical embedded software development.

A proof-of-concept compiler for *stC*, known as **stcc**, has been written in Ada 2022. It acts as a transpiler that translates *stC* source code into SPARK Ada 2022 [1] source code that can be formally verified with GNATprove. The code of **stcc** is available in GitHub at <https://github.com/jgrivera67/stcc>.

1.1 Language Overview

The main goal of *stC* is to make the timeless safe and secure software engineering principles of Ada/SPARK more accessible and appealing to embedded C software developers, by presenting them in the more familiar C syntax. The language enforces strong typing, supports design-by-contract, and transpiles to SPARK Ada so that the full power of GNATprove formal verification is available to every *stC* programmer.

- *stC* is a fully strongly-typed language. There is no automatic integer promotion or any other implicit type conversions.
- There are no built-in integer types in *stC*. All integer types must be explicitly declared, using language constructs to define range-based or modular-based integer types. The only built-in types are **bool** and **char**.

- Unlike C, *stC* allows the programmer to write completely pointer-free code in general. No explicit pointers are needed for ordinary data structures, and because the language does not support dynamic memory allocation, pointers are not needed for heap management either. Using pointers is the exception, not the norm.
- *stC* provides programming-by-contract constructs. Function contracts — preconditions (`[[pre]]`), postconditions (`[[post]]`), and type invariants (`[[type_invariant]]`) — are part of the signature of a function or type declaration, not just comments. Loop contracts — loop invariants (`[[loop_invariant]]`) and loop variants (`[[loop_variant]]`) — can be attached to any loop to prove termination and absence of runtime errors within the loop body. Data-flow contracts — global variable clauses (`[[global]]`) and information-flow dependencies (`[[depends]]`) — precisely capture which state a function reads or writes and how each output depends on the inputs. Because the transpiler emits all contracts as SPARK aspects, they are statically provable by GNATprove.
- Precise bit-level representation of hardware registers can be specified as part of the corresponding struct declarations, using `[[at]]`, `[[offset]]`, `[[bits]]`, and `[[volatile]]` attributes. This makes safe MMIO register access straightforward, without raw pointer casts or fragile `volatile` structs.
- Types can be qualified with logical predicates to support “liquid types” (*logically qualified* types).
- Arrays cannot implicitly be treated as pointers and they carry their length when passed as parameters to functions.
- There are no header files and no preprocessor.
- Declarations can be in any order. Symbols can be used before they are declared. In *stC* there is no need for function prototypes or forward declarations.
- A module has an explicit name specified at the beginning of the source file with the `module` keyword.
- A module can import declarations from other modules using their names. An import can be prefixed with `private` (analogous to Ada’s `private with`), meaning the imported module is only usable within the current module’s private section.
- A module can have private declarations that are not visible to other modules.
- A module can have child modules. A child module is named by using its parent module’s name followed by a dot as a prefix.

- A child module inherits all the public and private declarations of its ancestor modules. However, inherited private declarations cannot be used in the visible parts of the module’s public declarations.
- A child module can be declared *private* by prefixing its `module` declaration with the `private` keyword. A private child module is only accessible to its parent module and to sibling modules within the same module family; it cannot be imported by unrelated modules. This corresponds to Ada’s *private child package* concept.
- The language is case-sensitive. All reserved words are lower case.
- The language is block-structured. Blocks are delimited by curly braces.

1.2 `stcc`

`stcc` is implemented in Ada 2022 and acts as a transpiler: it translates *stC* source files into equivalent SPARK Ada 2022 source code. The generated code is intended to be fully compilable with GNAT and formally verifiable with GNATprove [1].

- For each *stC* file an Ada package specification file (`.ads`) and an Ada package body file (`.adb`) are generated. To compile these files with the GNAT compiler, the `-gnatX` flag must be used, to enable the curated subset of GNAT language extensions, which are used in the generated files [2].
- Line number from the original *stC* source file are included as end-of-line comments in the generated Ada code.

Chapter 2

Lexical Elements

2.1 Tokens

The lexical structure of *stC* consists of the following token types:

- Keywords (reserved words)
- Identifiers
- Literals (integer, floating-point, character, string, boolean)
- Operators
- Punctuation symbols
- Comments

2.2 Keywords

The following identifiers are reserved as keywords and cannot be used as identifiers:

<code>abstract_state</code>	<code>as</code>	<code>assert</code>	<code>auto</code>	<code>bit</code>
<code>bits</code>	<code>bool</code>	<code>break</code>	<code>case</code>	<code>char</code>
<code>compile_if</code>	<code>const</code>	<code>continue</code>	<code>convention</code>	<code>decimal</code>
<code>default</code>	<code>delta</code>	<code>depends</code>	<code>digits</code>	<code>do</code>
<code>else</code>	<code>enum</code>	<code>false</code>	<code>first</code>	<code>fixed</code>
<code>float</code>	<code>for</code>	<code>foreign</code>	<code>generic</code>	<code>global</code>
<code>goto</code>	<code>if</code>	<code>import</code>	<code>in</code>	<code>inout</code>
<code>last</code>	<code>loop_invariant</code>	<code>loop_variant</code>	<code>modular</code>	<code>module</code>
<code>offset</code>	<code>out</code>	<code>packed</code>	<code>post</code>	<code>pre</code>
<code>private</code>	<code>range</code>	<code>reads</code>	<code>refined_state</code>	<code>renames</code>
<code>return</code>	<code>size</code>	<code>sizeof</code>	<code>static</code>	<code>struct</code>
<code>subtype</code>	<code>switch</code>	<code>true</code>	<code>type</code>	<code>type_invariant</code>
<code>union</code>	<code>void</code>	<code>volatile</code>	<code>while</code>	<code>writes</code>

Note: `machine_width` is not a keyword but a built-in constant (see section on Built-in Constants).

2.3 Identifiers

An identifier consists of a letter followed by zero or more letters, digits, or underscores:

```
identifier  = letter (letter | digit | '_')*
letter      = 'a'..'z' | 'A'..'Z'
digit       = '0'..'9'
```

Identifiers are case-sensitive. Examples: `myVariable`, `Count_1`, `MAX_SIZE`

2.4 Literals

2.4.1 Integer Literals

stC supports three integer literal formats:

- **Decimal:** 0, 42, 1234, 1_000_000
- **Hexadecimal:** 0x0, 0x2A, 0x4D2, 0xDEAD_BEEF, 0x8000_0000
- **Binary:** 0b0, 0b101010, 0b1111_1111, 0b1010_0101

Both underscores (`_`) and apostrophes (`'`) can be used as digit separators to improve readability. Apostrophes are supported for compatibility with C.

Separators are allowed between digits but not at the beginning or end of a number, and not immediately after the base prefix (`0x` or `0b`). A literal must use only one separator style - mixing `_` and `'` in the same literal is an error.

Examples:

```
1_000_000          // Decimal with underscores
1'000'000          // Decimal with apostrophes (C compatibility)
0xDEAD_BEEF        // Hexadecimal with underscores
0xDEAD'BEEF        // Hexadecimal with apostrophes
0b1111_1111        // Binary with underscores
0b1111'1111        // Binary with apostrophes

// Invalid examples
// 0x8000_0'000    // Error: mixed separators
// 1000_           // Error: trailing separator
```

2.4.2 Floating-Point Literals

Floating-point literals have the form:

```

3.14
2.71828
1.0e10
6.022e23

```

2.4.3 Character Literals

Character literals are enclosed in single quotes:

```
'a'  'Z'  '0'  '\n'  '\t'  '\\\'  '\'
```

Escape sequences:

- `\n` – newline
- `\t` – tab
- `\r` – carriage return
- `\\` – backslash
- `\'` – single quote
- `\"` – double quote
- `\0` – null character

2.4.4 String Literals

String literals are enclosed in double quotes:

```

"Hello , World!"
"Line 1\nLine 2"
"Path: C:\\Users\\name"

```

2.4.5 Boolean Literals

The boolean type has two literals: `true` and `false`

2.5 Operators

stC provides the following operators, listed by precedence (highest to lowest):

Precedence	Operators	Associativity
16	. -> [] () (function call, cast)	Left
15	** (exponentiation)	Right
14	* (deref) & (address-of) sizeof ! ~ - (unary) + (unary)	Right
13	* / %	Left
12	+ -	Left
11	<< >>	Left
10	< > <= >= ..	Left
9	== != in	Left
8	& (bitwise and)	Left
7	^	Left
6		Left
5	&&	Left
4		Left
3	? : (ternary)	Right
2	= += -= *= /= %= &= = ^= <<= >>=	Right

2.6 Comments

stC supports single-line comments starting with `//` and extending to the end of the line:

```
// This is a comment
int x = 42; // Comment after code
```

2.7 Built-in Constants

stC provides platform-specific built-in constants that can be configured via compiler command-line options.

2.7.1 Platform Constants

The following built-in constants are available:

- `machine_width` – The native word size in bits (e.g., 8, 16, 32, 64)

2.7.2 Compiler Options

The `machine_width` constant can be set using the command-line option:

```
stcc --machine-width=64 program.stc
```

Default value: 32 (suitable for most embedded platforms)

Example usage:

```
module stdint {  
    // Creates a modular type sized to the platform's native word  
    type modular 1 << machine_width address_t;  
}
```

Chapter 3

Examples

3.1 Simple Program

A minimal *stC* program:

```
import stdio as io;

module hello {

    foreign convention("C")
    int printf(in char* format);

    int main() {
        printf("Hello , World!\n");
        return 0;
    }
}
```

3.2 Scalar Types

3.2.1 Range Types

Range types define integer types with explicit minimum and maximum bounds. The compiler enforces these bounds, and runtime checks can verify that values stay within range.

Syntax:

```
type range <min-expr> .. <max-expr> <type-name>;
```

Examples:

```
// Integer types based on ranges
type range 0 .. 255 byte_t;
type range 0 .. 100 percentage_t;
```

```
type range -273 .. 1000 temperature_t;
type range -32768 .. 32767 int16_t;
```

```
percentage_t cpu_usage = 75;
temperature_t room_temp = 22;
```

Range types are ideal for:

- Values with natural limits (percentages, temperatures, indices)
- Representing subsets of integers with specific meaning
- Enabling compiler optimizations based on known bounds

3.2.2 Modular Types

Modular types define unsigned integer types with wraparound arithmetic modulo a specified value. When a value exceeds the modulus, it wraps around to zero. Modular types are perfect for implementing C-style unsigned integers and bit manipulation.

Syntax:

```
type modular <modulus-expr> <type-name>;
```

The modulus defines the range `[0, modulus-1]`. Arithmetic operations wrap around automatically.

Defining C Standard Unsigned Integer Types:

stC provides multiple equivalent ways to define standard unsigned integer types. You can use decimal literals, bit shift expressions, or exponentiation:

```
// uint8_t: 8-bit unsigned (0..255)
type modular 256 uint8_t;      // Using decimal literal
type modular 1 << 8 uint8_t    // Using bit shift
type modular 2 ** 8 uint8_t;    // Using exponentiation

// uint16_t: 16-bit unsigned (0..65535)
type modular 65_536 uint16_t;  // Using decimal literal
type modular 1 << 16 uint16_t; // Using bit shift
type modular 2 ** 16 uint16_t; // Using exponentiation

// uint32_t: 32-bit unsigned (0..4294967295)
type modular 4_294_967_296 uint32_t; // Using decimal literal
type modular 1 << 32 uint32_t;      // Using bit shift
type modular 2 ** 32 uint32_t;      // Using exponentiation

// uint64_t: 64-bit unsigned (0..18446744073709551615)
type modular 1 << 64 uint64_t;      // Using bit shift (most readable)
type modular 2 ** 64 uint64_t;      // Using exponentiation
```

Wraparound Behavior:

```
uint8_t counter = 255;
counter = counter + 1; // Wraps to 0

uint8_t value = 0;
value = value - 1;    // Wraps to 255
```

Modular types are ideal for:

- Bit manipulation and bitwise operations
- Hardware register access
- Cryptography and checksums
- Circular buffers and wraparound counters
- Compatibility with C unsigned integer types

3.2.3 Floating Point Types

Floating point types define real number types with specified decimal precision. They are similar to Ada floating point types and provide approximate representation of real numbers.

Syntax:

```
type float digits <precision> <type-name>;
type float digits <precision> range <min> .. <max> <type-name>;
```

The **digits** clause specifies the minimum number of decimal digits of precision. An optional **range** clause constrains the valid values.

Examples:

```
// Basic floating point types
type float digits 6 float32_t;      // At least 6 decimal digits
type float digits 15 float64_t;     // At least 15 decimal digits

// Constrained floating point types
type float digits 6 range -273.15 .. 1000.0 temperature_t;
type float digits 8 range 0.0 .. 100.0 percentage_t;
type float digits 10 range -180.0 .. 180.0 longitude_t;
type float digits 10 range -90.0 .. 90.0 latitude_t;

// Usage
temperature_t room_temp = 22.5;
percentage_t cpu_load = 87.3;
latitude_t current_lat = 37.7749;
longitude_t current_lon = -122.4194;
```

Floating point types are ideal for:

- Scientific computations requiring real numbers

- Measurements with decimal precision
- Geographic coordinates and spatial calculations
- Physics simulations

Note: The actual precision depends on the target platform’s floating point implementation. The `digits` value specifies the minimum required precision.

3.2.4 Fixed Point Types

Fixed point types define real number types with exact decimal representation using a fixed delta value. They provide better precision than floating point for certain ranges and are deterministic.

Ordinary Fixed Point Syntax:

```
type fixed delta <delta-value> range <min> .. <max> <type-name>;
```

The `delta` specifies the smallest representable difference between values. All values are exact multiples of the delta.

Examples:

```
// Ordinary fixed point types
type fixed delta 0.001 range 0.0 .. 5.0 voltage_t;      // mV precision
type fixed delta 0.01 range -100.0 .. 100.0 sensor_t;   // 10ms precision
type fixed delta 0.125 range 0.0 .. 1000.0 precise_t;    // 1/8 precision

// Usage
voltage_t supply_voltage = 3.300;  // Exactly 3.300V
sensor_t reading = 25.75;          // Exactly 25.75
precise_t measurement = 123.625;   // Exactly 123.625

// All operations maintain exact precision
voltage_t v1 = 1.500;
voltage_t v2 = 1.750;
voltage_t sum = v1 + v2;  // Exactly 3.250
```

Decimal Fixed Point Syntax:

```
type decimal delta <delta-value> digits <precision> <type-name>;
```

Decimal fixed point types are specifically designed for financial calculations where exact decimal arithmetic is required.

Examples:

```
// Decimal fixed point for financial applications
type decimal delta 0.01 digits 10 money_t;      // Pennies, up to 99999999.99
type decimal delta 0.001 digits 12 price_t;     // Mils precision
type decimal delta 0.0001 digits 8 interest_t;  // Basis points

// Usage
```



```

money_t account_balance = 1234.56;
money_t transaction = 42.99;
account_balance = account_balance + transaction; // Exactly 1277.55

price_t stock_price = 123.456;
interest_t rate = 0.0325; // 3.25%

```

Fixed point types are ideal for:

- Financial calculations requiring exact decimal arithmetic
- Embedded systems with no floating point hardware
- Control systems requiring deterministic behavior
- Currency and monetary values
- Sensor readings with known precision

Key Differences:

- **Ordinary fixed point:** Requires explicit range, optimized for specific value ranges
- **Decimal fixed point:** Range derived from digits, optimized for decimal calculations
- **Floating point:** Approximate representation, wider range, variable precision

3.2.5 Subtypes

Subtypes allow you to define constrained versions of existing types. A subtype is compatible with its base type but has additional range constraints. This is particularly useful for creating semantically meaningful types that share the same underlying representation.

Syntax:

```

subtype <base-type> range <min-expr> .. <max-expr> <subtype-name>;

```

Examples:

```

// Define base signed integer type
type range -2147483648 .. 2147483647 int_t;

// Natural numbers as a subtype of int_t (non-negative integers)
subtype int_t range 0 .. 2147483647 natural_t;

// Positive numbers as a subtype of int_t
subtype int_t range 1 .. 2147483647 positive_t;

// Percentage as a subtype of natural_t

```

```

subtype natural_t range 0 .. 100 percentage_t;

// Usage
int_t temperature = -10;      // Can be negative
natural_t count = 42;        // Cannot be negative
positive_t items = 1;        // Must be at least 1
percentage_t progress = 75;   // Limited to 0..100

// Subtype values are compatible with base type
int_t x = count;              // OK: natural_t is a subtype of int_t
natural_t y = temperature;    // Runtime check: may fail if negative

```

Key characteristics:

- Subtypes are compatible with their base type
- Values can be assigned between base type and subtype (with runtime checks)
- Subtypes inherit operations from their base type
- Subtype ranges must be within the base type's range
- Subtypes can be derived from other subtypes (transitive relationship)

3.2.6 Structure Types

```

type range -1000 .. 1000 coordinate_t;
type struct {
    coordinate_t x;
    coordinate_t y;
} point_t;

type range 0 .. 150 age_t;
type struct {
    char* name;
    age_t age;
    bool is_employed;
} person_t;

person_t p;
p.name = "Alice";
p.age = 30;
p.is_employed = true;

```

3.2.7 Structure Types with Type Invariants

Type invariants specify constraints that must always hold for instances of a type:

```

type range -1000 .. 1000 coordinate_t;
type struct {
    coordinate_t x;
    coordinate_t y;
} point_t [[type_invariant(x * x + y * y <= 1000000)]];

// This ensures all point_t values satisfy the distance constraint
// from origin: sqrt(x^2 + y^2) <= 1000

```

3.2.8 Union Types

```

type range -2147483648 .. 2147483647 int_t;
type union {
    int_t int_value;
    char* string_value;
    bool bool_value;
} value_t;

```

3.2.9 Enumeration Types

```

type enum {
    red,
    green = 10,
    blue,
    yellow = 20
} color_t;

color_t my_color = red;

```

3.2.10 Bit Fields and Memory-Mapped I/O Registers

stC supports precise bit-level layout control for struct fields using Ada/SPARK-style representation clauses. This feature is essential for embedded systems programming, particularly for Memory-Mapped I/O (MMIO) register definitions where exact hardware layout must be specified.

Field-Level Bit Attributes

Field attributes specify the exact byte offset and bit position for each struct field using `[[offset(...)]]` combined with `bit(...)` or `bits(...)`:

```

type modular 1 << 8 uint8_t;

type struct {
    bool enable [[offset(0) bit(0)]];
    bool interrupt_enable [[offset(0) bit(1)]];

```

```
uint8_t mode [[offset(0) bits(2..4)]]; // 3 bits
uint8_t reserved [[offset(0) bits(5..7)]]; // 3 bits
} control_reg_t [[packed]];
```

Attribute Syntax:

- `[[offset(N) bit(M)]]` – Field at byte N, single bit M (for booleans)
- `[[offset(N) bits(M..P)]]` – Field at byte N, bits M through P (for multi-bit fields)
- `[[volatile]]` – Field represents hardware register (suppress optimization)

Important Rules:

- **offset** is **required** when using **bit** or **bits** (avoids C bit field ambiguity)
- Either **all** fields in a struct have explicit `[[offset(...)]]` attributes or **none** do
- No mixing of explicit and implicit layout in the same struct
- **Struct WITHOUT offset/bits attributes:** uses C layout (compiler applies normal struct packing rules)
- **Struct WITH offset/bits attributes:** uses explicit Ada representation (programmer controls every bit position)

Type-Level Attributes

Type attributes apply to the entire struct:

- `[[packed]]` – No padding between fields, minimal size
- `[[at(address)]]` – Map type to specific memory address (for MMIO)
- `[[volatile]]` – All fields are volatile (hardware registers)

Complete MMIO Register Example

Example of a 32-bit GPIO control register:

```
type modular 1 << 32 uint32_t;
type range 0 .. 7 uint3_t;
type range 0 .. 15 uint4_t;
type range 0 .. 255 uint8_t;

// GPIO Control Register (32-bit at 0x40000000)
type struct {
    bool port_enable [[offset(0) bit(0)]]; // Bit 0
```

```

    bool output_enable [[offset(0) bit(1)]];    // Bit 1
    bool pull_up [[offset(0) bit(2)]];          // Bit 2
    bool pull_down [[offset(0) bit(3)]];        // Bit 3
    uint4_t drive_strength [[offset(0) bits(4..7)]]; // Bits 4–7
    uint8_t reserved1 [[offset(1) bits(0..7)]]; // Bits 8–15
    uint8_t pin_value [[offset(2) bits(0..7)]]; // Bits 16–23
    uint8_t reserved2 [[offset(3) bits(0..7)]]; // Bits 24–31
} gpio_control_t [[packed]] [[at(0x40000000)]] [[volatile]];

// Access the memory-mapped register
void gpio_init() {
    gpio_control_t* gpio = 0x40000000;
    gpio->port_enable = true;
    gpio->output_enable = true;
    gpio->drive_strength = 7; // Maximum drive strength
    gpio->pull_up = false;
    gpio->pull_down = false;
}

```

Maps to Ada/SPARK:

The stC bit field definition above transpiles to Ada with exact representation clauses:

```

type GPIO_Control_T is record
    Port_Enable      : Boolean;
    Output_Enable    : Boolean;
    Pull_Up          : Boolean;
    Pull_Down        : Boolean;
    Drive_Strength    : Unsigned_4;
    Reserved1         : Unsigned_8;
    Pin_Value         : Unsigned_8;
    Reserved2         : Unsigned_8;
end record;

for GPIO_Control_T use record
    Port_Enable      at 0 range 0 .. 0;
    Output_Enable    at 0 range 1 .. 1;
    Pull_Up          at 0 range 2 .. 2;
    Pull_Down        at 0 range 3 .. 3;
    Drive_Strength    at 0 range 4 .. 7;
    Reserved1         at 1 range 0 .. 7;
    Pin_Value         at 2 range 0 .. 7;
    Reserved2         at 3 range 0 .. 7;
end record;

for GPIO_Control_T'Size use 32;

GPIO : GPIO_Control_T
    with Address => System.Storage_Elements.To_Address (16#40000000#),

```

```
    Volatile;
```

Regular Structs Without Bit Fields

For regular structs that don't require precise bit layout, simply omit the bit field attributes:

```
type struct {
    bool flag;           // Standard C layout
    uint8_t count;       // Standard alignment
    uint32_t value;      // Standard alignment
} config_t;
```

3.3 Arrays

```
type range 0 .. 99 index_t;
index_t[100] numbers;
numbers[0] = 42;
```

```
// Multi-dimensional arrays
index_t[10][10] m;
m[0][0] = 1;
```

3.4 Functions

3.4.1 Attributes and Aspects

In *stC*, C23-style attributes (using `[[...]]` syntax) are the mechanism for specifying aspects, similar to Ada/SPARK aspects. This includes:

- **Function contracts:** `[[pre(...)]], [[post(...)]], [[global(reads/writes => (...))]]`
- **Function attributes:** `[[no_return]]`
- **Loop contracts:** `[[loop_invariant(...)]], [[loop_variant(...)]]`
- **Type attributes:** `[[type_invariant(...)]], [[packed]], [[at(...)]]`
- **Variable attributes:** `[[at(...)]]`

The `[[...]]` syntax provides a familiar C-like notation while enabling the same level of specification and verification found in Ada/SPARK.

3.4.2 Simple Function

```
int_t add(in int_t a, in int_t b) {
    return a + b;
}
```

3.4.3 Function with Contracts

Contracts in *stC* use C23-style attribute syntax with double square brackets `[[...]]`. This is *stC*'s way of implementing aspects, similar to SPARK contracts.

```
type range -2147483648 .. 2147483647 int_t;
subtype int_t range 0 .. 2147483647 natural_t;

natural_t divide(in natural_t a, in int_t b)
    [[pre(b != 0)]]
    [[post(result * b <= a && result * b + b > a)]]
{
    return a / b;
}
```

3.4.4 Function with Global-Effect Contracts

Global-effect contracts specify which global variables a function reads or writes, similar to SPARK's `Global` aspect:

```
type modular 1 << 32 uint32_t;
uint32_t global_counter;
uint32_t global_sum;

uint32_t increment_counter()
    [[global(reads => (global_counter), writes => (global_sum))]]
    [[pre(global_counter < 1000)]]
    [[post(global_sum > 0)]]
{
    global_sum = global_sum + global_counter;
    return global_sum;
}

// Function that only reads globals (no writes)
uint32_t compute_total()
    [[global(reads => (global_counter, global_sum))]]
{
    return global_counter + global_sum;
}
```

3.4.5 Function with Parameter Modes

```
void swap(inout int_t a, inout int_t b) {
    int_t temp = a;
    a = b;
    b = temp;
}

// Usage
int_t x = 10;
int_t y = 20;
swap(x, y); // x=20, y=10
```

3.5 Control Flow

3.5.1 If Statement

```
type range -2147483648 .. 2147483647 int_t;

int_t absolute(in int_t x) {
    if (x < 0) {
        return -x;
    } else {
        return x;
    }
}
```

3.5.2 While Loop

```
type range 0 .. 99 index_t;
index_t i = 0;

while (i < 10) {
    // Loop body
    i = i + 1;
}
```

3.5.3 While Loop with Loop Contracts

Loop contracts specify invariants and variants for formal verification:

```
type range 0 .. 99 index_t;
type modular 1 << 32 uint32_t;

uint32_t sum_array(index_t [] arr, uint32_t length) {
    uint32_t sum = 0;
```



```
index_t i = 0;

while (i < length)
    [[ loop_invariant(i <= length)]]
    [[ loop_variant(length - i)]]
    {
        sum = sum + arr[i];
        i = i + 1;
    }

return sum;
}
```

3.5.4 For Loop

```
type range 0 .. 99 index_t;

for (index_t i = 0; i < 10; i = i + 1) {
    // Loop body
}
```

3.5.5 Do-While Loop

```
type range 0 .. 99 index_t;
index_t i = 0;

do {
    // Loop body
    i = i + 1;
} while (i < 10);
```

3.5.6 Switch Statement

```
type enum {
    monday, tuesday, wednesday,
    thursday, friday, saturday, sunday
} day_of_week_t;

day_of_week_t day = monday;

switch (day) {
    case monday:
    case tuesday:
    case wednesday:
    case thursday:
    case friday:
```

```

        // Weekday
        break;
    case saturday:
    case sunday:
        // Weekend
        break;
    default:
        // Invalid
        break;
}

```

3.6 Identifier Attributes

stC supports SPARK-style identifier attributes using the apostrophe operator. These attributes provide type and array information at compile time:

3.6.1 Range Type Attributes

```

type range 0 .. 99 index_t;
type modular 100 array_100_t[100];

uint32_t process_array(array_100_t arr) {
    uint32_t sum;
    index_t i;

    sum = 0;

    // 'first gives the first valid index value
    i = index_t 'first;

    // 'last gives the last valid index value
    while (i <= index_t 'last) {
        sum = sum + arr[i];
        i = i + 1;
    }

    return sum;
}

```

3.6.2 Array Attributes

Like Ada/SPARK, identifier attributes also work with arrays to query their index ranges and length:

```

type range 0 .. 99 index_t;
type modular 256 byte_t;

```

```

byte_t buffer[50];

void process_buffer() {
    // For arrays, 'first' returns the first index
    index_t first_idx = buffer 'first';    // Returns 0

    // For arrays, 'last' returns the last index
    index_t last_idx = buffer 'last';      // Returns 49

    // For arrays, 'length' returns the number of elements
    index_t len = buffer 'length';        // Returns 50

    // For arrays, 'range' returns the index range
    for (index_t i in buffer 'range') {
        buffer[i] = 0;
    }
}

```

3.6.3 Available Attributes

- `identifierfirst` – For range types: returns the first value of the range. For arrays: returns the first index
- `identifierlast` – For range types: returns the last value of the range. For arrays: returns the last index
- `identifierlength` – For arrays: returns the number of elements (equivalent to `last - first + 1`)
- `identierrange` – For range types: returns the full range. For arrays: returns the index range
- `identifiersize` – Returns the size in bytes

3.7 Sizeof Operator

The `sizeof` operator returns the size in bytes of a type or expression, similar to C:

```

type range 0 .. 255 byte_t;
type struct {
    byte_t x;
    byte_t y;
} point_t;

// sizeof with type name
uint32_t size1 = sizeof(point_t);    // Size of the type

```

```

uint32_t size2 = sizeof(byte_t);    // Size of byte_t

// sizeof with expression
point_t p;
uint32_t size3 = sizeof(p);          // Size of the variable
uint32_t size4 = sizeof(p.x);        // Size of the field

```

Note: Both `identifiersize'` and `sizeof(identifier)` return size in bytes. Use `sizeof` when you need the size of an expression or when the C-style syntax is preferred.

3.8 Expressions

3.8.1 Ternary Conditional

```

type range -2147483648 .. 2147483647 int_t;

int_t max(in int_t a, in int_t b) {
    return (a > b) ? a : b;
}

```

3.8.2 Range Operator

```

type range 0 .. 100 percentage_t;
percentage_t score = 85;

if (score in 90 .. 100) {
    // Grade A
} else {
    if (score in 80 .. 89) {
        // Grade B
    }
}

```

3.9 Pointers

```

type range -2147483648 .. 2147483647 int_t;

int_t x = 42;
int_t* ptr = &x;    // Address-of operator
int_t value = *ptr; // Dereference operator

// Pointer to struct
type struct {
    int_t x;
}

```

```

    int_t y;
} point_t;

point_t p;
point_t* p_ptr = &p;
p_ptr->x = 10;      // Arrow operator
p_ptr->y = 20;

```

3.10 Function Signature Types

A *function signature type* names the signature (parameters and return type) of a function. Variables and parameters of a function signature type hold a reference to a callable function; the compiler passes them by reference implicitly — no explicit pointer syntax is required.

```

type range -2147483648 .. 2147483647 int_t;

// Declare a function signature type with no parameters, returning void
type void timer_cb_t();

// Declare a function signature type with parameters, returning void
type void binary_op_cb_t(in int_t a, in int_t b);

// Declare a function signature type returning a value
type int_t int_transform_t(in int_t x);

// Use as a parameter — the compiler passes the function reference implicitly
void register_timer(in timer_cb_t callback, in uint32_t period_ms);

// Assign a named function to a function-signature-type variable
void my_handler() { }
timer_cb_t cb = my_handler;

// Call through the variable like a normal function
cb();

// Pass directly as an argument
register_timer(my_handler, 100);

```

3.11 Module Example

```

module geometry {

    type range -1000 .. 1000 coordinate_t;

    type struct {

```

```

    coordinate_t x;
    coordinate_t y;
} point_t;

coordinate_t distance_squared(in point_t p1, in point_t p2) {
    coordinate_t dx = p2.x - p1.x;
    coordinate_t dy = p2.y - p1.y;
    return dx * dx + dy * dy;
}

private

// Private helper function
coordinate_t square(in coordinate_t x) {
    return x * x;
}
}

```

3.12 Child Modules

A *public* child module is visible to any module that imports it. It inherits the public and private declarations of all its ancestor modules.

```

// File: geometry.circle.stc
module geometry.circle {

    // Inherits point_t and coordinate_t from parent module geometry

    type range 0 .. 1000 radius_t;

    type struct {
        point_t center;
        radius_t radius;
    } circle_t;

    bool point_in_circle(in point_t p, in circle_t c) {
        return distance_squared(p, c.center) <= square(c.radius);
    }

}

```

3.12.1 Private Child Modules

A *private* child module is prefixed with the **private** keyword. It can only be imported by its parent module and by sibling modules within the same module family.

Unrelated modules cannot import it. This corresponds directly to Ada's *private child package*.

```
// File: geometry.internal.stc
// Private child — only geometry and geometry.* siblings can import this
private module geometry.internal {

    // Inherits point_t and coordinate_t from parent module geometry

    // Internal helper not exposed to users of the geometry module family
    coordinate_t clamp(in coordinate_t v,
                      in coordinate_t lo,
                      in coordinate_t hi) {
        if (v < lo) return lo;
        if (v > hi) return hi;
        return v;
    }
}

// File: geometry.circle.stc (sibling — allowed to import the private child)
private import geometry.internal; // private: only used in implementation

module geometry.circle {

    type range 0 .. 1000 radius_t;

    coordinate_t clamped_x(in point_t p, in coordinate_t lo, in coordinate_t hi) {
        return geometry.internal.clamp(p.x, lo, hi);
    }
}

// File: app.stc (unrelated module — NOT allowed to import the private child)
import geometry.circle; // OK: public child
// import geometry.internal; // ERROR: private child, not accessible here

module app {

}
```

3.12.2 Private Imports

A **private import** restricts an imported module to the private section of the importing module. It is analogous to Ada's **private with** clause and is useful when an import is an implementation detail that should not leak into the public interface.

```
// File: board.stc
import cpu; // public: cpu types appear in the public interface
```

```

private import cpu.multicore; // private: only used in the implementation

module board {

    const cpu.cpu_model_t cpu_model = cpu.cortex_a72; // OK: cpu is public

    void start_secondary_cpu(in cpu.secondary_cpu_core_id_t cpu_id,
                             in cpu.address_t entry_point_address);

    private

    void start_core_impl() {
        // OK: cpu.multicore is accessible here (private section)
        cpu.multicore.wake(0);
    }
}

```

3.13 Compile-Time Conditionals

The `compile_if` statement allows for conditional compilation based on constant expressions evaluated at compile time. This is useful for platform-specific code or conditional features.

3.13.1 Platform-Specific Types

```

module platform_types {

    // Define types based on platform word size
    compile_if machine_width == 64 {
        type range 0 .. (1 << 63) - 1 unsigned_long_t;
        type range -(1 << 63) .. (1 << 63) - 1 signed_long_t;
    } else {
        type range 0 .. (1 << 31) - 1 unsigned_long_t;
        type range -(1 << 31) .. (1 << 31) - 1 signed_long_t;
    }

    // Use underscores for readability in large numbers
    compile_if machine_width >= 32 {
        type modular 0x1-0000-0000 uint32_t; // 4 billion
    }
}

```


3.13.2 Numeric Literal Separators

Both underscores and apostrophes can be used as digit separators in numeric literals to improve readability. Apostrophes are supported for compatibility with C.

```
module numeric_separators {

    // Underscore separators
    type range 0 .. 1_000_000 million_t;
    type modular 0x8000_0000 half_uint32_t;
    type modular 0b1111_1111 byte_mask_t;

    // Apostrophe separators (for C compatibility)
    type range 0 .. 1'000'000 million_alt_t;
    type modular 0xDEAD'BEEF hex_value_t;

    // Note: A literal must use only one separator style
    // type modular 0x8000_0'000 invalid_t; // Error: mixed separators

}
```

3.14 Depends Contracts

Information-flow contracts specify which outputs depend on which inputs, enabling formal verification of data-flow properties. This maps directly to SPARK's **Depends** aspect.

```
module depends_example {

    // swap: each output depends on both inputs.
    // x => (x, y) is stC's way of expressing SPARK's x =>+ y
    // (output depends on inputs AND its own initial value).
    void swap(inout int32_t x, inout int32_t y)
        [[depends(x => (x, y), y => (x, y))]]
    {
        int32_t tmp = x;
        x = y;
        y = tmp;
    }

    // clear: result is independent of all inputs
    void clear(out int32_t result)
        [[depends(result => void)]]
    {
        result = 0;
    }

    // copy: dst depends solely on src
```

```

void copy(out int32_t dst, in int32_t src)
  [[depends(dst => src)]]
{
  dst = src;
}
}

```

Note: SPARK’s $\Rightarrow+$ syntax (“*output depends on inputs and on its own prior value*”) is expressed in *stC* by including the output variable in its own input list. For example, $x \Rightarrow (x, y)$ is equivalent to SPARK’s $x \Rightarrow+ y$. The explicit form is preferred because it is unambiguous and familiar to C programmers.

3.15 Abstract State and Refined State

Large modules can hide their internal variables behind an abstract state name, enabling callers to write information-flow contracts at the module interface level without knowing the concrete implementation variables. This maps to SPARK’s **Abstract_State** and **Refined_State** aspects.

```

module Server [[abstract_state(State)]] {

  // Public interface refers only to the abstract state name.
  void init()  [[global(writes => (State))]] {}
  void push(in uint8_t val)
              [[global(writes => (State))]] {}
  uint32_t count()
              [[global(reads => (State))]] { return 0; }

  private

  // Private variables are tagged with the abstract state they refine.
  uint8_t  buffer[64] [[refined_state(State)]];
  uint32_t head       [[refined_state(State)]];
  uint32_t tail       [[refined_state(State)]];

}

```

3.16 Generic Modules

stC supports parameterised modules using C-style generic formal parameter syntax. Generics are instantiated via **import** with named actual parameters.

3.16.1 Type and Function Formals

```

generic(type T, bool less_than(in T a, in T b))
module Sort_Utills {

    void sort(inout T arr[], in uint32_t len) {}
    bool is_sorted(in T arr[], in uint32_t len) { return true; }

}

    Instantiate with named actual parameters using ==>:
import Sort_Utills(T ==> int32_t, less_than ==> int32_less) as Int_Sort;

```

3.16.2 Type and Const Formals

```

generic(type T, const uint32_t Capacity)
module Fixed_Queue {

    type struct {
        T items[Capacity];
        uint32_t head;
        uint32_t tail;
        uint32_t count;
    } queue_t;

    void init(inout queue_t q) {}
    bool enqueue(inout queue_t q, in T item){ return false; }
    bool dequeue(inout queue_t q, out T item){ return false; }

}

```

Instantiate as:

```

import Fixed_Queue(T ==> uint8_t, Capacity ==> 64) as Byte_Queue;

```

Appendix A

stC Grammar

```
<module> ::=
  (("private"? "import" <module-name> (<generic-actual-part>)?
    ("as" <module-identifier>)? ";")*
  (<generic-formal-part>)?
  ("private"? "module" <module-name> (<module-attributes>)?
    "{"
  <declaration>*
  ("private" <declaration>+)? "}")

<generic-formal-part> ::=
  "generic" "(" <generic-formal-parameter> (","
    <generic-formal-parameter>)* ")"

<generic-formal-parameter> ::=
  "type" <type-identifier> |
  "const" <type-identifier> <constant-identifier> |
  <type-identifier> <function-identifier>
  "(" (<param-declaration> ("," <param-declaration>)*)? ")"

<function-identifier> ::= "<identifier>"

<generic-actual-part> ::=
  "(" <generic-actual-parameter> (","
    <generic-actual-parameter>)* ")"

<generic-actual-parameter> ::=
  "<identifier>" "=" ("<identifier>" | <constant-expression>)

<module-attributes> ::= ("[" <module-attribute> "]" )+

<module-attribute> ::=
  "abstract_state" "(" <identifier> ("," <identifier>)* ")"
```

```

<module-name> ::= "<identifier>" ( "." "<identifier>" ) *

<module-identifier> ::= "<identifier>"

<declaration> ::=
  <constant-declaration> |
  <type-declaration> |
  <function-sig-type-declaration> |
  <subtype-declaration> |
  <global-variable-declaration> |
  <function-declaration> |
  <foreign-variable-declaration> |
  <foreign-function-declaration> |
  <compile-if-declaration>

<constant-declaration> ::=
  "const" <type-identifier> <constant-declarator> "="
    <constant-expression> ";" |
  "const" "type" <type-declarator> <type-identifier>
    <constant-identifier>
    "=" <constant-expression> ";"

<constant-declarator> ::=
  ("*")? <constant-identifier>
    ("[" " "]" ("[" <constant-integer-expression> "]" )*) |
  "[" <constant-integer-expression> "]" *

<constant-identifier> ::= "<identifier>"

<type-declaration> ::=
  "type" <type-declarator> <type-identifier>
    (<type-attributes>)? ";"

<function-sig-type-declaration> ::=
  "type" (<return-type-identifier> | "void") <type-identifier>
  "(" (<param-declaration> ( "," <param-declaration> )*)? ")" ";"

<return-type-identifier> ::= <type-identifier>

<subtype-declaration> ::=
  "subtype" <type-identifier> "range"
    <constant-integer-expression>
    ".." <constant-integer-expression> <subtype-identifier>
    (<type-attributes>)? ";"

<type-identifier> ::= "<identifier>" ( "." "<identifier>" ) *

<subtype-identifier> ::= "<identifier>"

```

```

<type-declarator> ::=
    "range" <constant-integer-expression> ".."
        <constant-integer-expression> |
    "modular" <constant-integer-expression> |
    "float" "digits" <constant-integer-expression>
        ("range" <constant-expression> ".."
            <constant-expression>)? |
    "fixed" "delta" <constant-expression> "range"
        <constant-expression> ".." <constant-expression> |
    "decimal" "delta" <constant-expression> "digits"
        <constant-integer-expression> |
    ("struct" | "union") "{" (<struct-field-declaration> ";")+
        "}" |
    "enum" "{" <enum-entry> ("," <enum-entry>)* "}"

<struct-field-declaration> ::=
    <type-identifier> <struct-field-declarator>
        (<field-attributes>)? ("=" <constant-expression>)?

<struct-field-declarator> ::=
    ("*")? <struct-field-identifier>
        ("[" <constant-integer-expression> "]" ) *

<struct-field-identifier> ::= "<identifier>"

<field-attributes> ::=
    ("[" <field-attribute> "]" ) +

<field-attribute> ::=
    "offset" "(" <constant-integer-expression> ")"
    "bit" "(" <constant-integer-expression> ")" |
    "bits" "(" <constant-integer-expression> ".."
        <constant-integer-expression> ")" |
    "volatile"

<enum-entry> ::=
    <enum-entry-identifier> ("=" <constant-integer-expression>)?

<enum-entry-identifier> ::= "<identifier>"

<global-variable-declaration> ::=
    ("const" | "volatile")? <type-identifier>
        <global-variable-declarator>
        ("=" <constant-expression>)? (<variable-attributes>)? ";"

<global-variable-declarator> ::=
    ("*")? <global-variable-identifier>

```

```

    ("[" <constant-integer-expression> "]" ) *

<global-variable-identifier> ::= "<identifier>"

<function-declaration> ::=
    ("void" | <type-identifier> ("*")?) <function-identifier>
    "(" (<param-declaration> ("," <param-declaration>)* )? ")"
    (<function-attributes>)?
    <statement-block>

<statement-block> ::=
    "{ " (<local-variable-declaration> | <statement>)* "}"

<local-variable-declaration> ::=
    ("volatile")? <type-identifier> <local-variable-declarator>
    ("=" <expression>)? (<variable-attributes>)? ";"

<local-variable-declarator> ::=
    ("*")? <local-variable-identifier> ("[" <index-expression>
    "]" ) *

<local-variable-identifier> ::= "<identifier>"

<statement> ::=
    <assignment> |
    <void-function-call> |
    <if-statement> |
    <while-statement> |
    <for-statement> |
    <do-while-statement> |
    <switch-statement> |
    <statement-block> |
    <return-statement> |
    <compile-if-statement>

<assignment> ::= <variable-reference> "=" <expression>

<void-function-call> ::= <void-function-reference> "("
    (<function-call-argument> ("," <function-call-argument>)* )?
    ")"

<void-function-reference> ::=
    <void-function-identifier> |
    <void-function-pointer-expression>

<if-statement> ::= "if" "(" <boolean-expression> ")"
    <statement-block> ("else" <statement-block>)?

```

```

<while-statement> ::= "while" "(" <boolean-expression> ")"
    (<loop-attributes>)? <statement-block>

<do-while-statement> ::= "do" <statement-block> "while" "("
    <boolean-expression> ")" ";"

<for-statement> ::=
    "for" "(" (<local-variable-declaration> | <assignment>)? ";"
    <boolean-expression>? ";"
    <assignment>? ")" (<loop-attributes>)? <statement-block> |
    "for" "(" <type-identifier> <local-variable-identifier> "in"
    <expression> ")"
    (<loop-attributes>)? <statement-block>

<switch-statement> ::=
    "switch" "(" <expression> ")" "{"
    <switch-case>*
    (<default-case>)?
    "}"

<switch-case> ::= "case" <constant-expression> ":" <statement>*

<default-case> ::= "default" ":" <statement>*

<return-statement> ::= "return" (<expression>)? ";"

<compile-if-statement> ::=
    "compile_if" <constant-expression> "{"
    <statement>*
    "}" ("else" "{" <statement>* "}")?

<compile-if-declaration> ::=
    "compile_if" <constant-expression> "{"
    <declaration>*
    "}" ("else" "{" <declaration>* "}")?

<param-declaration> ::=
    (<parameter-mode>)? <type-identifier> <param-declarator>

<parameter-mode> ::= "in" | "out" | "inout"

<param-declarator> ::=
    ("*")? <param-identifier> ("[" "]" ("["
    <constant-integer-expression> "]" )*)?

<param-identifier> ::= "<identifier>"

<function-call-argument> ::= <expression>

```



```

<foreign-variable-declaration> ::=
    "foreign" "convention" "(" <string-literal> ")"
    <type-identifier> <global-variable-declarator> ";"

<foreign-function-declaration> ::=
    "foreign" "convention" "(" <string-literal> ")"
    ("void" | <type-identifier> ("*")?) <function-identifier>
    "(" (<param-declaration> ("," <param-declaration>)*)? ")" ";"

<expression> ::= <ternary-expression>

<ternary-expression> ::=
    <logical-or-expression> ("?" <expression> ":" <expression>)?

<logical-or-expression> ::=
    <logical-and-expression> ("||" <logical-and-expression>)*

<logical-and-expression> ::=
    <bitwise-or-expression> ("&&" <bitwise-or-expression>)*

<bitwise-or-expression> ::=
    <bitwise-xor-expression> ("|" <bitwise-xor-expression>)*

<bitwise-xor-expression> ::=
    <bitwise-and-expression> ("^" <bitwise-and-expression>)*

<bitwise-and-expression> ::=
    <equality-expression> ("&" <equality-expression>)*

<equality-expression> ::=
    <relational-expression> (("==" | "!=" | "in")
    <relational-expression>)*

<relational-expression> ::=
    <shift-expression> ((" "< | ">" | "<=" | ">=" | "..")
    <shift-expression>)*

<shift-expression> ::=
    <additive-expression> ((" "<< | ">>") <additive-expression>)*

<additive-expression> ::=
    <multiplicative-expression> (("+" | "-")
    <multiplicative-expression>)*

<multiplicative-expression> ::=
    <power-expression> (("*" | "/" | "%") <power-expression>)*

```

```

<power-expression> ::=
    <unary-expression> ("**" <unary-expression>)*

<unary-expression> ::=
    <postfix-expression> |
    <unary-operator> <unary-expression> |
    "sizeof" "(" (<type-identifier> | <expression>) ")"

<unary-operator> ::= "+" | "-" | "!" | "~" | "*" | "&"

<postfix-expression> ::=
    <primary-expression>
    (( "[" <expression> "]" ) |
     "(" (<function-call-argument> ("," <function-call-argument>)* )? ")" ) |
     "." <struct-field-identifier> ) |
     "->" <struct-field-identifier> ) |
     "' " <identifier-attribute> ))*

<identifier-attribute> ::= "first" | "last" | "length" |
    "range" | "size"

<primary-expression> ::=
    <identifier> |
    <literal> |
    "(" <expression> ")" |
    <type-cast-expression> |
    "{" <expression> ("," <expression>)* "}"

<type-cast-expression> ::= <type-identifier> "(" <expression>
    ")"

<boolean-expression> ::= <expression>

<index-expression> ::= <expression>

<constant-expression> ::= <expression>

<constant-integer-expression> ::=
    "<binary-integer-literal>" |
    "<decimal-integer-literal>" |
    "<hexadecimal-integer-literal>" |
    <constant-identifier> |
    <integer-unary-op> <constant-integer-expression> |
    <constant-integer-expression> <integer-binary-op>
    <constant-integer-expression>

<integer-unary-op> ::= "+" | "-" | "~"

```

```

<integer-binary-op> ::=
    "+" | "-" | "*" | "/" | "%" | "**" | "<<" | ">>" |
    "&" | "|" | "^"

<literal> ::=
    "<binary-integer-literal>" |
    "<decimal-integer-literal>" |
    "<hexadecimal-integer-literal>" |
    "<floating-point-literal>" |
    "<character-literal>" |
    "<string-literal>" |
    "<boolean-literal>"

<boolean-literal> ::= "true" | "false"

<variable-reference> ::=
    <variable-identifier> ( "." <struct-field-identifier> ( "["
        <index-expression> "]" ) ) * |
    <variable-identifier> ( "[" <index-expression> "]" ) *

<variable-identifier> ::= "<identifier>"

<void-function-identifier> ::= "<identifier>"

<variable-attributes> ::=
    ( "[" <variable-attribute> "]" ) +

<variable-attribute> ::=
    "at" ( " <constant-expression> " ) |
    "refined_state" ( " "<identifier>" " )

<function-attributes> ::=
    <function-attribute> +

<function-attribute> ::=
    "[" ( <function-contract> | "no_return" ) "]"

<function-contract> ::=
    <precondition> | <postcondition> | <global-clause> |
    <depends-clause>

<depends-clause> ::=
    "depends" ( " "<depends-item> ( "," <depends-item> ) * " )

<depends-item> ::=
    <depends-output> "=>" <depends-rhs>

```

```

<depends-output> ::=
  "<identifier>" | "(" "<identifier>" ("," "<identifier>")* ")"

<depends-rhs> ::=
  "void" | "<identifier>" | "(" "<identifier>" (","
    "<identifier>")* ")"

<precondition> ::= "pre" "(" <boolean-expression> ")"

<postcondition> ::= "post" "(" <boolean-expression> ")"

<global-clause> ::=
  "global" "(" <global-effect> ("," <global-effect>)* ")"

<global-effect> ::=
  ("reads" | "writes") "=>" "(" <identifier> (","
    <identifier>)* ")"

<loop-attributes> ::=
  <loop-attribute>+

<loop-attribute> ::=
  "[" <loop-contract> "]"

<loop-contract> ::=
  <loop-invariant> | <loop-variant>

<loop-invariant> ::= "loop_invariant" "(" <boolean-expression>
  ")"

<loop-variant> ::= "loop_variant" "(" <expression> ")"

<type-attributes> ::=
  ("[" <type-attribute> "]" )+

<type-attribute> ::=
  "packed" | "at" "(" <constant-expression> ")" |
  "type_invariant" "(" <boolean-expression> ")"

```

Bibliography

- [1] John W. McCormick, Peter C. Chapin, *Building High Integrity Applications with SPARK*, Cambridge University Press, 2015
<https://www.amazon.com/Building-High-Integrity-Applications-SPARK/dp/1107040736>
- [2] AdaCore, “GNAT Reference Manual”, chapter 17, version 26.0w
https://docs.adacore.com/gnat_rm-docs/html/gnat_rm/gnat_rm/gnat_language_extensions.html